MIT 8103 Advanced Database Systems

Oluwanisola Dele-Adelusi

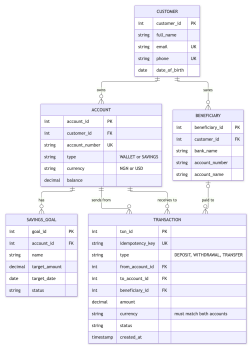
Student ID: 301910982

Case Study:

This case study is a personal savings first mobile banking system called Vant, aimed at young Nigerian customers. The database contains tables to record and manage customer, account, transactions and savings goals. This includes multiple operations such as creating accounts, making deposits, withdrawing, transferring funds, funding goals and viewing statements.

Portfolio 1:

Designed a relational database which has 5 tables. Tables have primary keys to identify data and foreign keys to maintain referential integrity. Constraints have been implemented to ensure the business rules are followed (unique and CHECK constraints). Composite foreign keys are used to ensure both accounts match the currency of the transaction. This database is in 3NF. There are 16 auto PASS/FAIL tests to test valid data. There are also 12 intentional invalid operations.



Portfolio 1: ER diagram

Portfolio 2:

Used EXPLAIN (ANALYZE, BUFFERS) to evaluate 5 complex queries from 205,000 synthetic transactions. Using 2 standard indexes and 1 partial index reduced the run time of Q1 to 0.16ms from 10.94ms and Q5 to 0.06ms from 10.27ms. This significantly reduced the amount of buffer pages being read. There was very little improvement in Q2-4 and this was justified using technical explanations. Testing the write throughput revealed there was an increase in latency when maintaining indexes of 1.5-2x.

Portfolio 3:

Demonstrated COMMIT and ROLLBACK. Automatically rolled back if a business rule was broken. Reproduced a race condition with a lost update anomaly by performing 2 withdrawals of

800 when the balance was only 1000 which led to the balance being incorrectly updated to 200. This was resolved using pessimistic locking by using SELECT ... FOR UPDATE to lock the row. This caused the second transaction to wait and be rejected due to insufficient funds. Compared READ COMMITTED and REPEATABLE READ. Fixed a deadlock by locking the accounts in a specific order. Used an idempotent transfer to reject any duplicate keys.

Portfolio 4:
Used MongoDB to store my data. As this is a document database, it allowed for flexible fields. Financial data was stored in PostgreSQL. Created 2 collections. An activity feed collection which contained 23,553 documents. These documents were extracted from the PostgreSQL transactions. Created a KYC collection with documents of different nested structures. Each document contains BVN, NIN, Passport, Liveness score (Selfie) and Proof of address. Added a $jsonSchema validator to reject any invalid documents. Created an index on customerId and createdAt to reduce the number of documents searched to 5 from 23,553. Created an aggregation to add up the totals. Added a unique constraint on pgTxnId to stop duplicates. Using a reconciliation script I realised the MongoDB database is only eventually consistent with PostgreSQL. This could be solved using a transactional outbox or change data capture.

Portfolio 5:
Created a primary PostgreSQL database with 1 streaming replica using Docker. Demonstrated asynchronous replication by slowing down the replication with a 3 second lag. When reading the balance straight after writing it was 1100, not 1200. Simulated a failover by disconnecting the replica. Made 3 deposits of 100 and promoted the replica. The new primary took 0.3 seconds to accept writes but had lost 300. This demonstrated the trade off between Recovery Point Objective and Recovery Time Objective. Hash partitioned the transactions into 4 partitions. There was even distribution with between 34,110 and 35,657 rows. Discussed synchronous replication. On reconnect, the old primary was fenced. Designed a cloud solution using Amazon RDS with a standby in another availability zone. Also created a managed MongoDB replica set (not deployed).

GitHub Repository:

https://github.com/adelusisola/Dele-Adelusi_Oluwanisola_301910982
This is a public repository, so can be opened by the assessor. There are SQL/NoSQL scripts, execution plans for queries, screenshots and READMEs for each portfolio. Everything is named appropriately.

Technical Reflection:
The most challenging part of this portfolio was Portfolio 3. Initially I thought that a CHECK constraint would be enough to ensure there are enough funds. However, when trying to perform 2 withdrawals at the same time, both passed the constraint and the balance became 200. I could

see why using row locking would help avoid this issue. By using SELECT ... FOR UPDATE to resolve the issue, I noticed there was a deadlock when trying to perform 2 opposite transfers. To resolve this, I always locked the accounts in ID order. This meant I learnt that just because you solve 1 issue doesn't mean there aren't any others. Every change should be tested.

In Portfolio 2, the indexes only helped with queries that were reading few rows. Sometimes the timings were quite noisy. I learnt to look at how many pages were being read. In Portfolio 4 and 5, I learnt that by using MongoDB and replicas there are some trade offs. Although there are benefits with flexibility and availability, there can be issues with data duplication. With asynchronous replication there can be lost transactions that have been acknowledged during a failover. I used NUMERIC(14,2) to store the money values. This is an exact decimal type and avoids any rounding errors. Many payment systems will store the value in the smallest currency, e.g. kobo, using integers. Both methods avoid rounding errors. However, I went for using NUMERIC as it is readable when querying and forces the value to be 2 decimal places.

Overall I learnt that it is important to understand the trade offs when designing databases. For financial balances and transactions I would use PostgreSQL with strict integrity rules. Other data stores would be used for workloads that can tolerate eventual consistency.

References:

Docker. (n.d.). *Docker documentation*. https://docs.docker.com/

Mermaid. (n.d.). *Mermaid CLI*. https://github.com/mermaid-js/mermaid-cli

MongoDB, Inc. (n.d.). *MongoDB manual (version 7.0)*. https://www.mongodb.com/docs/v7.0/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/

ThisDay. (2023, October 25). *Vant founder Akolade unveils vision to revolutionize personal finance in Nigeria*. https://www.thisdaylive.com/2023/10/25/vant-founder-akolade-unveils-vision-to-revolutionize-personal-finance-in-nigeria/